

A decorative graphic on the left side of the slide consisting of several overlapping hexagons in orange, teal, light blue, lime green, and red, arranged in a staggered, honeycomb-like pattern.

제11장 PCA, 결정트리

학습 목표

- PCA 알고리즘을 이해한다.
- 결정 트리를 이해한다.

SECTION 11-3 주성분 분석(1)

◦ 차원과 차원 축소

- ⑩ 특성은 데이터가 가진 속성. 과일 사진의 경우 10,000개의 픽셀은 10,000개의 특성이 있는 셈
- ⑩ 머신러닝에서는 이런 특성을 차원(dimension)이라고도 함
- ⑩ 10,000개의 특성은 결국 10,000개의 차원이라는 것인데 이 차원을 줄일 수 있다면 저장 공간을 크게 절약

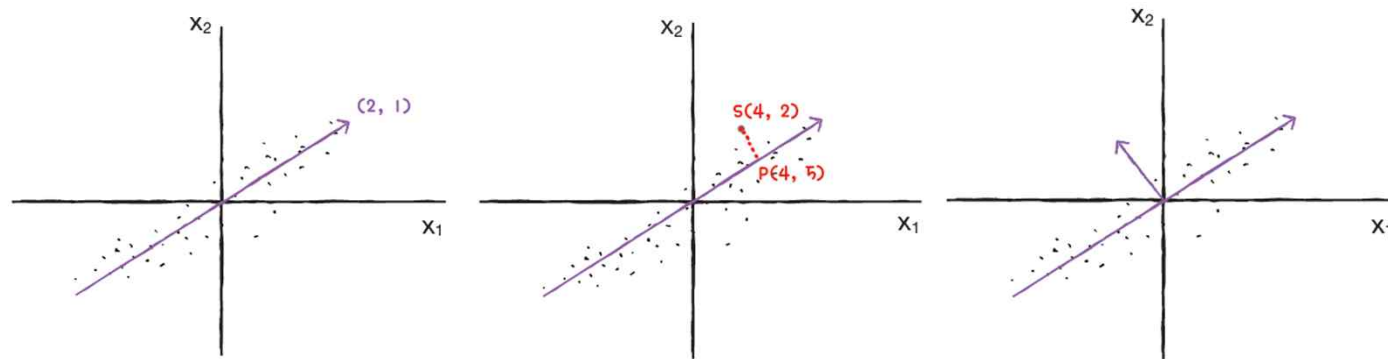
▪ 차원 축소(dimensionality reduction) 알고리즘

- ⑩ 차원 축소는 데이터를 가장 잘 나타내는 일부 특성을 선택하여 데이터 크기를 줄이고 지도 학습 모델의 성능을 향상시킬 수 있는 방법
- ⑩ 또한 줄어든 차원에서 다시 원본 차원(예를 들어 과일 사진의 경우 10,000개의 차원)으로 손실을 최대한 줄이면서 복원할 수도 있음
- ⑩ 주성분 분석(principal component analysis, PCA)은 대표적인 차원 축소 알고리즘

SECTION 11-3 주성분 분석(2)

◦ 주성분 분석 소개

- ⑩ 주성분 분석(PCA)은 데이터에 있는 분산이 큰 방향을 찾는 것
- ⑩ 분산이 큰 방향을 찾은 직선이 원점에서 출발한다면 두 원소로 이루어진 벡터로 표현 가능
- ⑩ 주성분 벡터의 원소 개수는 원본 데이터셋에 있는 특성 개수와 같음
- ⑩ 원본 데이터는 주성분을 사용해 차원을 줄일 수 있음
- ⑩ 주성분은 원본 차원과 같고 주성분으로 바꾼 데이터는 차원이 줄어듦



▲ [노트] 실제로 사이킷런의 PCA 모델을 훈련하면 자동으로 특성마다 평균값을 빼서 원점에 맞춰 줌. 따라서 우리가 수동으로 데이터를 원점에 맞추는 필요가 없음

SECTION 11-3 주성분 분석(3)

PCA 클래스

- ⑩ 이전 절과 마찬가지로 과일 사진 데이터를 다운로드하여 넘파이 배열로 적재

```
!wget https://bit.ly/fruits_300 -O fruits_300.npy
import numpy as np
fruits = np.load('fruits_300.npy')
fruits_2d = fruits.reshape(-1, 100*100)
```

- ⑩ PCA 클래스로 주성분 분석

```
from sklearn.decomposition import PCA
pca = PCA(n_components=50)
pca.fit(fruits_2d)
```

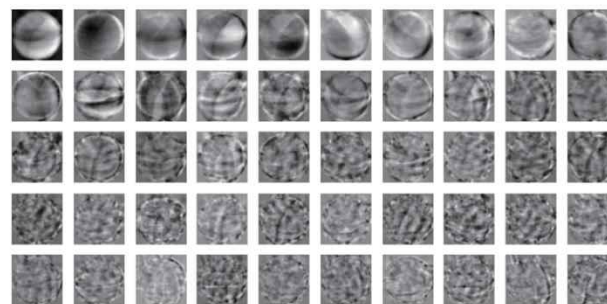
- ⑩ 배열의 크기 확인

```
print(pca.components_.shape) → (50, 10000)
```

- ⑩ draw_fruits() 함수를 사용해서 주성분 그리기

```
draw_fruits(pca.components_.reshape(-1, 100, 100)) →
```

draw_fruits(pca.components_.reshape(-1, 100, 100))
pca.components_ → (50, 10000)
reshape(-1, 100, 100) → 50개의 100×100 이미지
이건 주성분 자체를 시각화한 것! (데이터 아님)



SECTION 11-3 주성분 분석(4)

◦ PCA 클래스

⑩ 원본 데이터를 주성분에 투영하여 특성의 개수를 10,000개에서 50개로 줄일 수 있음

⑩ PCA의 transform() 메서드를 사용해 원본 데이터의 차원을 50으로 축소

```
print(fruits_2d.shape)
```



(300, 10000)

```
fruits_pca = pca.transform(fruits_2d)  
print(fruits_pca.shape)
```



(300, 50)

▲ 데이터가 1/200으로 축소

SECTION 11-3 주성분 분석(5)

◦ 원본 데이터 재구성

- ⑩ `inverse_transform()` 메서드를 사용하여, 앞서 50개의 차원으로 축소한 `fruits_pca` 데이터를 전달해 10,000개의 특성을 복원

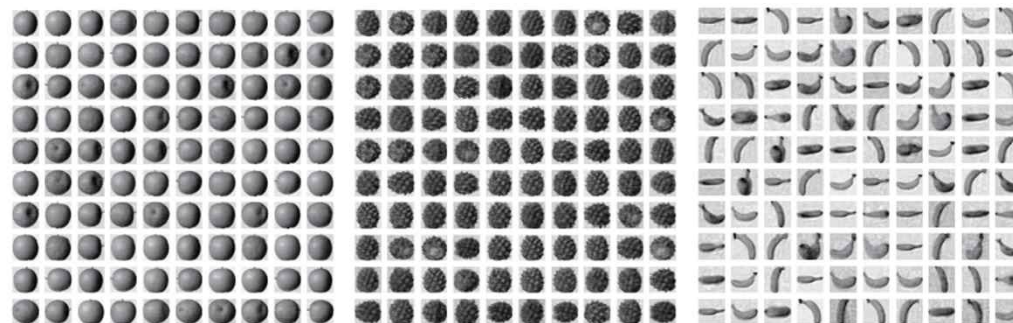
```
fruits_inverse = pca.inverse_transform(fruits_pca)
print(fruits_inverse.shape)
```

→ (300, 10000)

- ⑩ 100×100 크기로 바꾸어 100개씩 나누어 출력

```
fruits_reconstruct = fruits_inverse.reshape(-1, 100, 100)
for start in [0, 100, 200]:
    draw_fruits(fruits_reconstruct[start:start+100])
    print("\n")
```

→



SECTION 11-3 주성분 분석(6)

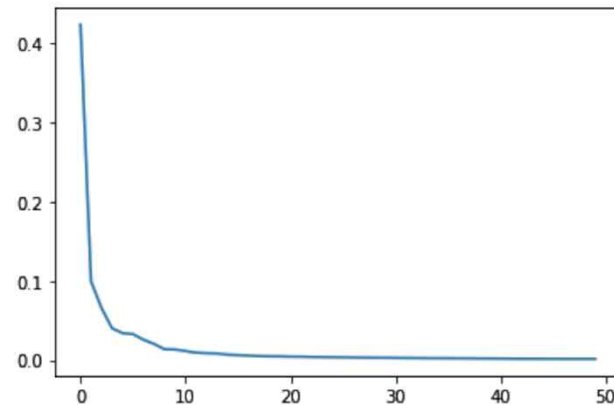
◦ 설명된 분산(explained variance)

- ⑩ 주성분이 원본 데이터의 분산을 얼마나 잘 나타내는지 기록한 값
- ⑩ PCA 클래스의 `explained_variance_ratio_`에 각 주성분의 설명된 분산 비율이 기록
 - 첫 번째 주성분의 설명된 분산이 가장 큼
 - 분산 비율을 모두 더하면 50개의 주성분으로 표현하고 있는 총 분산 비율을 얻을 수 있음

```
print(np.sum(pca.explained_variance_ratio_)) → 0.9215190262621741
```

- ⑩ 맷플롯립의 `plot()` 함수로 설명된 분산을 그래프로 출력

```
plt.plot(pca.explained_variance_ratio_)  
plt.show()
```



SECTION 11-3 주성분 분석(7)

◦ 다른 알고리즘과 함께 사용하기

⑩ 과일 사진 원본 데이터와 PCA로 축소한 데이터를 지도 학습에 적용해 보고 차이점 관찰

⑩ 3개의 과일 사진을 분류를 위해 사이킷런의 LogisticRegression 모델 만들기

```
from sklearn.linear_model import LogisticRegression  
lr = LogisticRegression()
```

⑩ 사과를 0, 파인애플을 1, 바나나를 2로 지정하여, 100개의 0, 100개의 1, 100개의 2로 이루어진 타깃 데이터 만들기

```
target = np.array([0]*100 + [1]*100 + [2]*100)
```

⑩ cross_validate()로 교차 검증 수행

```
from sklearn.model_selection import cross_validate  
scores = cross_validate(lr, fruits_2d, target)  
print(np.mean(scores['test_score']))  
print(np.mean(scores['fit_time']))
```

→ 0.9966666666666667
0.9422160625457764

⑩ PCA로 축소한 fruits_pca를 사용했을 때와 비교

```
scores = cross_validate(lr, fruits_pca, target)  
print(np.mean(scores['test_score']))  
print(np.mean(scores['fit_time']))
```

→ 1.0
0.03256878852844238

▲ 50개의 특성만 사용했는데도 정확도가 100%이고
훈련 시간은 0.03초로 20배 이상 감소

SECTION 11-3 주성분 분석(8)

◦ 다른 알고리즘과 함께 사용하기

⑩ `n_components` 매개변수에 분산의 비율을 입력하여, 설명된 분산의 50%에 달하는 주성분을 찾도록

PCA 모델 만들기

```
pca = PCA(n_components=0.5)
pca.fit(fruits_2d)
```

⑩ 탐색한 주성분 개수 확인

```
print(pca.n_components_)
```

→ 2

⑩ 이 모델로 원본 데이터를 변환

```
fruits_pca = pca.transform(fruits_2d)
print(fruits_pca.shape)
```

→ (300, 2)

⑩ 교차 검증의 결과 확인

```
scores = cross_validate(lr, fruits_pca, target)
print(np.mean(scores['test_score']))
print(np.mean(scores['fit_time']))
```

→ 0.9933333333333334
0.04122166633605957

SECTION 11-3 주성분 분석(9)

- 다른 알고리즘과 함께 사용하기

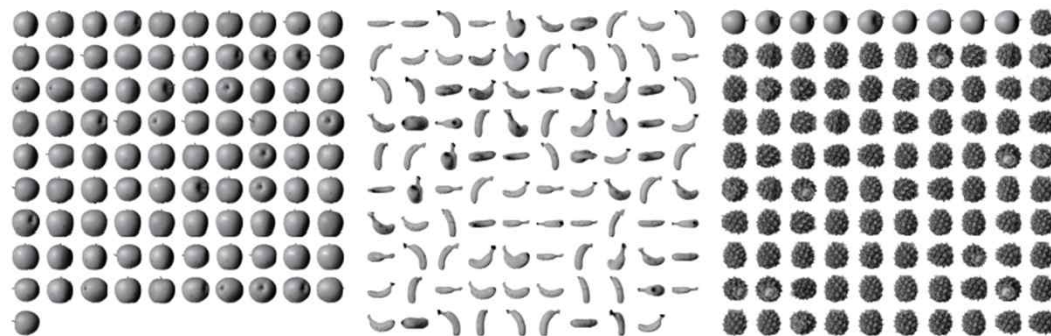
- ⑩ 차원 축소된 데이터를 사용해 k-평균 알고리즘으로 클러스터 찾기

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42)
km.fit(fruits_pca)
print(np.unique(km.labels_, return_counts=True))
```

→ (array([0, 1, 2], dtype=int32),
array([91, 99, 110]))

- ⑩ KMeans가 찾은 레이블을 사용해 과일 이미지 출력

```
for label in range(0, 3):
    draw_fruits(fruits[km.labels_ == label])
    print("\n")
```

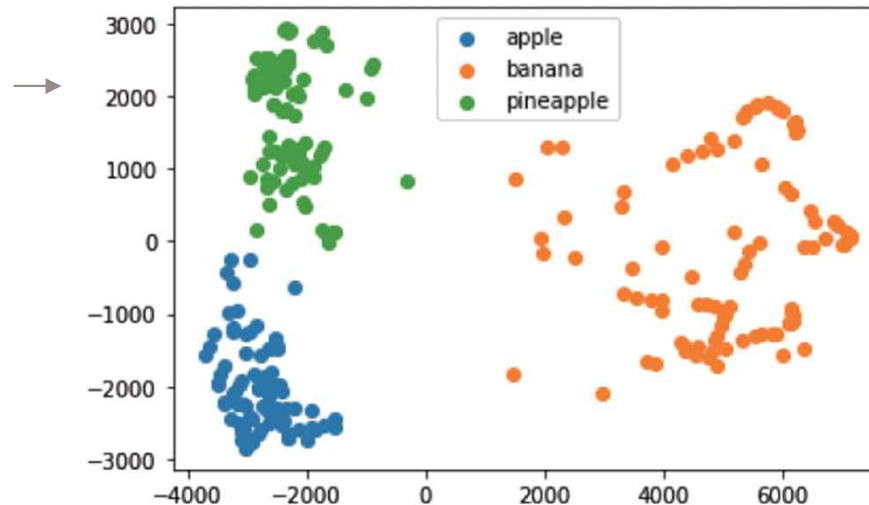


SECTION 11-3 주성분 분석(10)

- 다른 알고리즘과 함께 사용하기

- ⑩ 훈련 데이터의 차원을 줄이면 또 하나 얻을 수 있는 장점은 시각화
- ⑩ 3개 이하로 차원을 줄이면 화면에 출력하기 비교적 쉬움
- ⑩ fruits_pca 데이터는 2개의 특성이 있기 때문에 2차원으로 표현
- ⑩ 앞에서 찾은 km.labels_를 사용해 클러스터별로 나누어 산점도 그리기

```
for label in range(0, 3):  
    data = fruits_pca[km.labels_ == label]  
    plt.scatter(data[:,0], data[:,1])  
plt.legend(['apple', 'banana', 'pineapple'])  
plt.show()
```



SECTION 11-3 주성분 분석(11)

◦ 주성분 분석으로 차원 축소(문제해결 과정)

▪ 대표적인 비지도 학습 문제 중 하나인 차원 축소

⑩ 차원 축소를 사용하면 데이터셋의 크기를 줄일 수 있고 비교적 시각화하기 쉬움

⑩ 또 차원 축소된 데이터를 지도 학습 알고리즘이나 다른 비지도 학습 알고리즘에 재사용하여 성능을 높이거나 훈련 속도를 빠르게 만들 수 있음

▪ 학습

⑩ 사이킷런의 PCA 클래스를 사용해 과일 사진 데이터의 특성을 50개로 크게 축소

⑩ 특성 개수는 작지만 변환된 데이터는 원본 데이터에 있는 분산의 90% 이상을 표현
(이를 설명된 분산이라 함)

⑩ PCA 클래스는 자동으로 설명된 분산을 계산하여 제공하고, 주성분의 개수를 명시적으로 지정하는 대신 설명된 분산의 비율을 설정하여 원하는 비율만큼 주성분을 찾을 수 있음

⑩ PCA 클래스는 변환된 데이터에서 원본 데이터를 복원하는 메서드도 제공

⑩ 변환된 데이터가 원본 데이터의 분산을 모두 유지하고 있지 않다면 완벽하게 복원되지 않지만 적은 특성으로도 상당 부분의 디테일을 복원할 수 있음

SECTION 11-3 마무리(1)

◦ 키워드로 끝내는 핵심 포인트

- 차원 축소는 원본 데이터의 특성을 적은 수의 새로운 특성으로 변환하는 비지도 학습의 한 종류
 - ⑩ 차원 축소는 저장 공간을 줄이고 시각화하기 쉬움
 - ⑩ 또한 다른 알고리즘의 성능을 높일 수도 있음
- 주성분 분석은 차원 축소 알고리즘의 하나로 데이터에서 가장 분산이 큰 방향을 찾는 방법
 - ⑩ 이런 방향을 주성분이라고 함
 - ⑩ 원본 데이터를 주성분에 투영하여 새로운 특성을 만들 수 있음
 - ⑩ 일반적으로 주성분은 원본 데이터에 있는 특성 개수보다 적음
- 설명된 분산은 주성분 분석에서 주성분이 얼마나 원본 데이터의 분산을 잘 나타내는지 기록한 것
 - ⑩ 사이킷런의 PCA 클래스는 주성분 개수나 설명된 분산의 비율을 지정하여 주성분 분석을 수행할 수 있음

SECTION 11-3 마무리(2)

- 핵심 패키지와 함수

- scikit-learn

- ⑩ PCA: 주성분 분석을 수행하는 클래스

- n_components: 주성분의 개수를 지정. 기본값은 None으로 샘플 개수와 특성 개수 중에 작은 것의 값을 사용
 - random_state: 넘파이 난수 시드 값을 지정
 - components_: 훈련 세트에서 찾은 주성분이 저장
 - explained_variance_: 설명된 분산이 저장
 - explained_variance_ratio_: 설명된 분산의 비율이 저장
 - inverse_transform() 메서드: transform() 메서드로 차원을 축소시킨 데이터를 다시 원본 차원으로 복원

SECTION 11-3 확인 문제

1. 샘플 개수가 1,000개이고 특성 개수는 100개인 데이터셋이 있다. 즉 이 데이터셋의 크기는 (1000, 100). 이 데이터를 사이킷런의 PCA 클래스를 사용해 10개의 주성분을 찾아 변환했을 때, 변환된 데이터셋의 크기는 얼마일까?

- ① (1000, 10)
- ② (10, 1000)
- ③ (10, 10)
- ④ (1000, 1000)

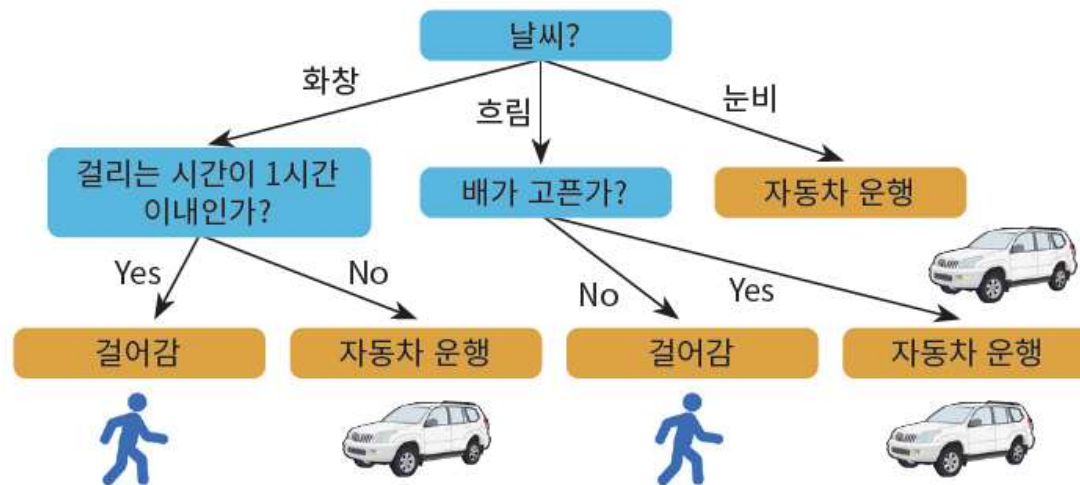
2. 위 문제에서 설명된 분산이 가장 큰 주성분은 몇 번째인가?

- ① 첫 번째 주성분
- ② 다섯 번째 주성분
- ③ 열 번째 주성분
- ④ 알 수 없음

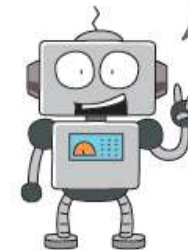


11.4 의사 결정 트리

- 의사 결정 트리(DT: Decision Trees)는 통계, 데이터 마이닝, 기계학습에 사용되는 지도 학습 방법의 일종이다. 회귀, 분류, 다중 출력까지 가능하다
- 예를 들어서 점심을 먹으러 갈 때, 운전을 할 것인지 건강을 위하여 걸어갈 것인지를 결정하려고 한다



질문을 하면서 질문의
대답에 따라서 입력을
분류하는 알고리즘입
니다.



의사 결정 트리의 장단점

- 의사 결정 트리는 인간의 사고를 모방하기 때문에 이해하고 해석하기 쉽다.
- 숫자 및 범주 데이터를 모두 처리할 수 있다.
- 데이터를 분류하는 논리의 흐름을 시각적으로 볼 수 있다.
- 의사 결정 트리의 단점은 지나치게 복잡한 트리를 생성할 수 있다는 점이다.
- 최적의 의사결정트리를 찾는 것은 **NP-complete** 문제

의사 결정 트리의 원리

- 아래 그림과 같이 2개의 특징을 이용하여 입력을 3개의 부류로 분류하려고 한다. 질문을 하나 하게 되면 이것은 그래프 상에서 직선을 그려서 분류하는 것과 같다.

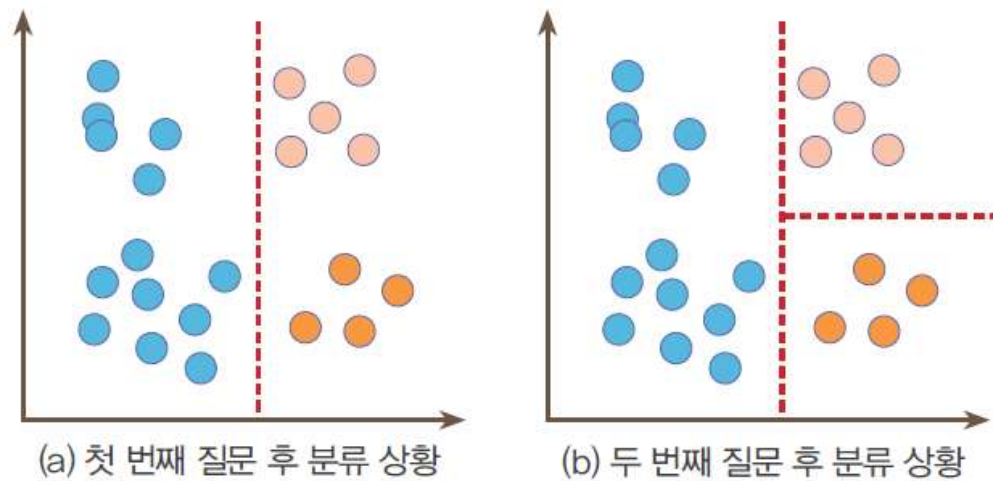


그림 11-16 의사 결정 트리의 원리

불순도(Impurity)란?

- 결정 트리는 노드를 분할할 때, 자식 노드가 더 순수(pure)해지도록 기준을 선택
- 순수함: 한 클래스가 압도적으로 많은 상태
- 불순도: 클래스가 얼마나 섞여 있는지 수치화
- → 불순도가 낮을수록 더 좋은 분할!

엔트로피 (Entropy) → ID3 알고리즘

- 공식:

- $\text{Entropy}(S) = -\sum p_i \log_2(p_i)$
- p_i : 클래스 i 의 비율
- 값의 범위: 0 (완전 순수) ~ 1 (완전 균등 분포)

- 정보 이득 (Information Gain)

- $\text{IG}(A) = \text{Entropy}(\text{부모}) - \sum (\text{가중 평균} \times \text{Entropy}(\text{자식}))$
- 엔트로피 감소량이 클수록 좋은 속성 선택

- ID3 (Iterative Dichotomiser 3) 특징

- 엔트로피 + 정보 이득 사용
- 다진 분할 가능 (multi-way split)
연속형 변수 불가 (전처리 필요)
결측치 처리 어려움

지니 계수 (Gini Index) → CART 알고리즘

- 공식:
 - $Gini(S) = 1 - \sum p_i^2$
 - 값의 범위: 0 (완전 순수) ~ 0.5 (이진 분류에서 완전 균등)
 - 계산이 더 빠름 (로그 연산 없음)
- 지니 감소량 (**Gini Gain**)
 - $Gini(\text{부모}) - \sum (\text{가중 평균} \times Gini(\text{자식}))$
- **CART** 특징
 - 이진 분할만 가능
 - 연속형 변수 가능
 - 회귀 트리 지원 (MSE)
 - 결측치 처리 용이

ID3 vs CART 비교

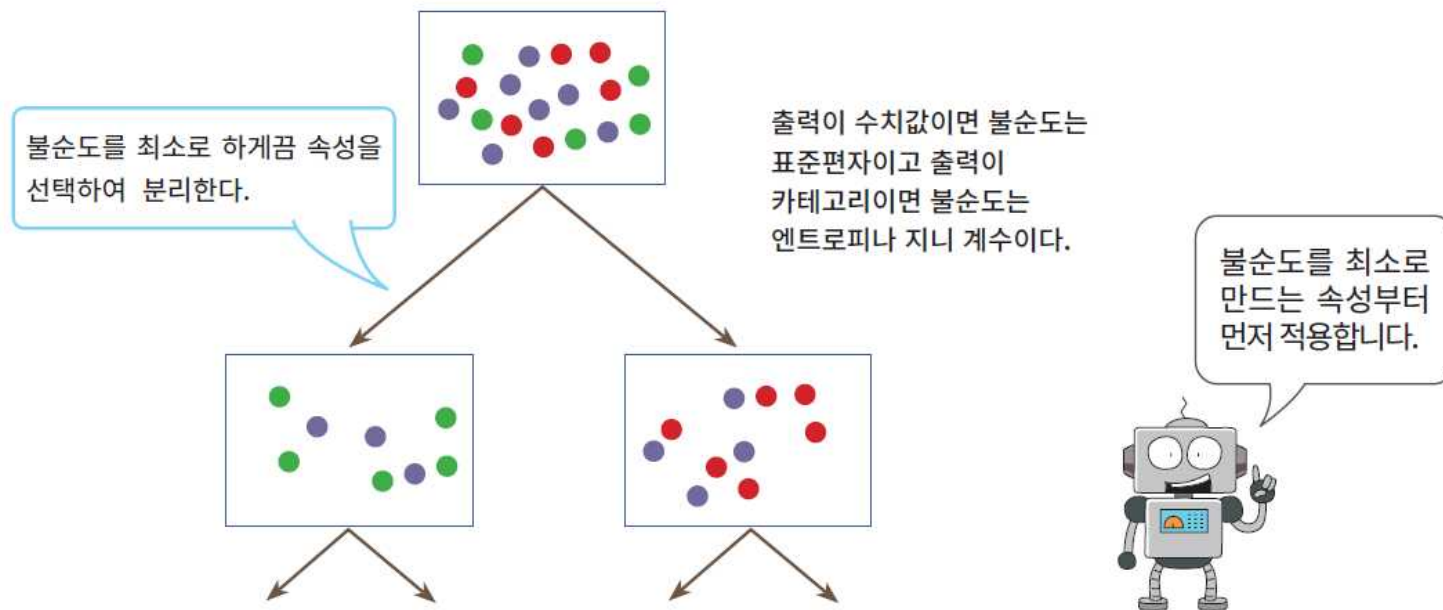
항목	ID3 (엔트로피)	CART (지니 계수)
불순도 지표	엔트로피 + 정보 이득	지니 계수
분할 방식	다진 분할	이진 분할만
연속형 변수	불가	가능
계산 속도	느림 (로그)	빠름
회귀 지원	불가	가능 (MSE)
결측치 처리	어려움	쉬움
대표 구현	C4.5	scikit-learn, XGBoost

결론

- 엔트로피를 쓰면 → **ID3**
- 지니 계수를 쓰면 → **CART**
- ID3는 정보 이론 기반(엔트로피)으로 범주형 데이터에 강함
- 현대 ML에서는 CART 기반 알고리즘이 주로 사용됨
(Random Forest, XGBoost 등)

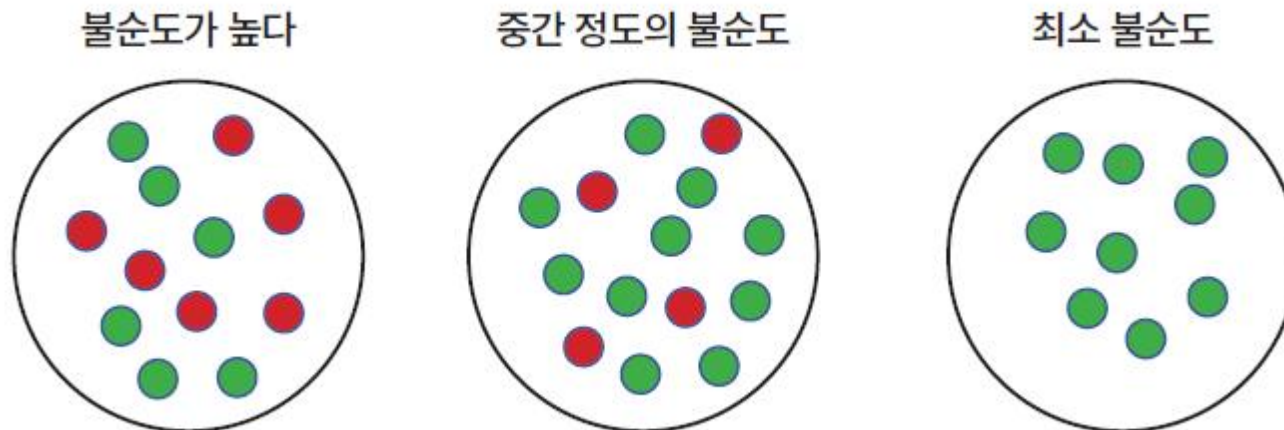
어떻게 트리를 구축하는 것일까?

- 의사 결정 트리를 만들기 위해서는 어떤 질문을 먼저 할 것인지를 결정하여야 한다.
- 변별력이 좋은 질문을 먼저 해야 한다. 분리된 결과의 불순도가 낮으면 변별도가 좋은 질문이다. 이것은 일종의 탐욕적인 알고리즘의 일종이다.



불순도의 지표

- 불순도로서 어떤 지표를 사용하느냐에 따라 알고리즘은 달라진다.
 - NP-complete
- 예를 들어서 엔트로피(Entropy)를 사용한다면 ID3와 같은 알고리즘이 되고 지니 계수(Gini Index)를 사용한다면 CART 알고리즘이 된다.
- 엔트로피란 무질서의 척도라고 할 수 있다.



엔트로피

- 엔트로피의 수학적 수식은 다음과 같다.

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

- 단순화를 위해 양성 클래스와 음성 클래스라는 두 개의 클래스만 있다고 가정해 보자 데이터 세트에 총 100개의 데이터 포인트가 있고 30개는 양성 클래스에 속하고 70개가 음성 클래스에 속하는 경우, p_+ 는 3/10이고 p_- 는 7/10이 된다. 위의 공식을 이용하여 엔트로피를 계산해보면 다음과 같다.

$$-\frac{3}{10} \times \log_2 \left(\frac{3}{10} \right) - \frac{7}{10} \times \log_2 \left(\frac{7}{10} \right) \approx 0.88$$

sklearn을 이용한 의사 결정 트리

- 위의 데이터 세트는 아주 간단하다. (0, 0)-> 0이고 (1, 1)->1이다. 학습이 끝난 후에 모델을 사용하여 (2, 2)가 어디에 속할지를 예측해보자.

```
>>> from sklearn import tree
>>> X = [[0, 0], [1, 1]]
>>> Y = [0, 1]
>>> clf = tree.DecisionTreeClassifier()
>>> clf = clf.fit(X, Y)
```

```
>>> clf.predict([[2., 2.]])
array([1])
```

예제: 의사 결정 트리를 이용한 붓꽃 분류

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, metrics
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_graphviz

iris = datasets.load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state = 50, test_size = 0.20)

clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

y_pred = clf.predict(X_train)
print("학습 데이터 정확도:", accuracy_score(y_true = y_train, y_pred = y_pred))

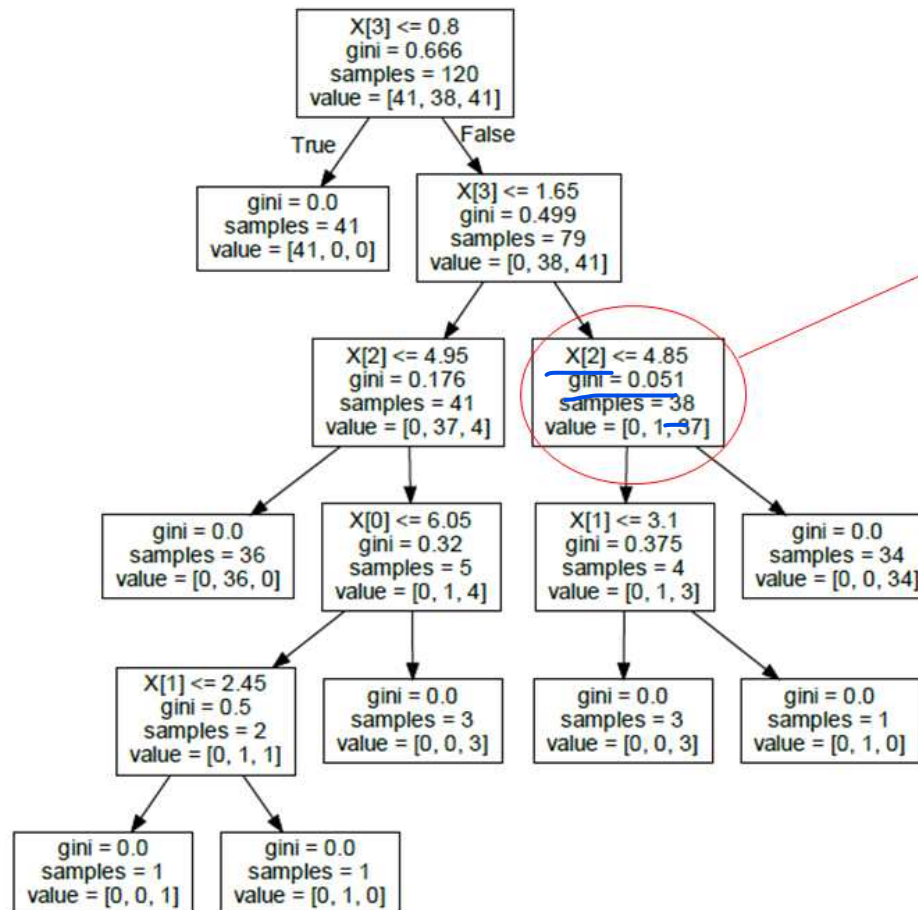
y_pred = clf.predict(X_test)
print("테스트 데이터 정확도:", accuracy_score(y_true = y_test, y_pred = y_pred))
```

학습 데이터 정확도: 1.0

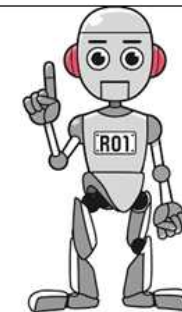
테스트 데이터 정확도: 0.9666666666666667

예제: 의사 결정 트리를 이용한 붓꽃 분류

```
from sklearn.tree import plot_tree
plot_tree(clf)
plt.show()
```

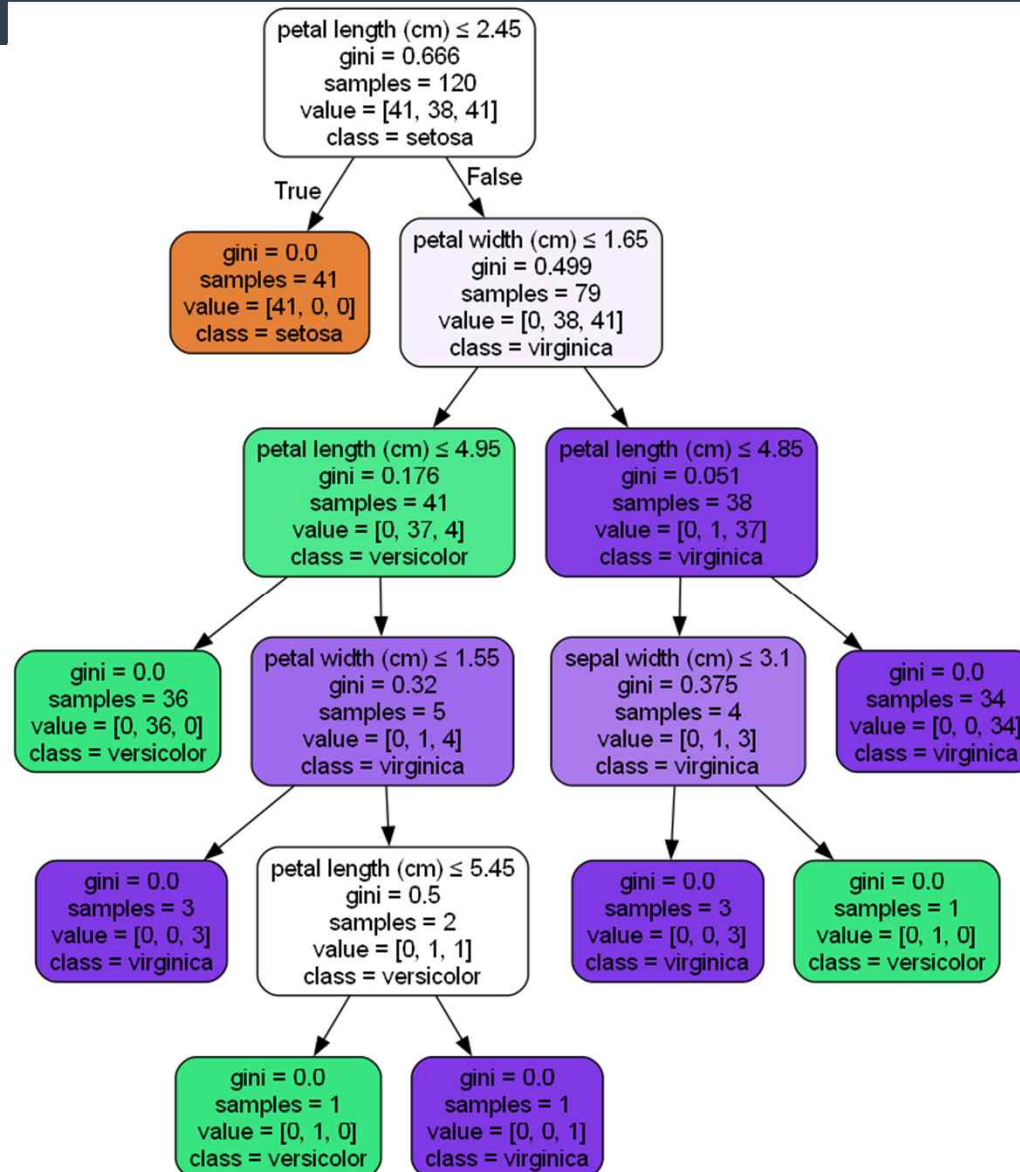


입력 특징 중에서 세번째 특징인 X[2]의 값이 4.85보다 작으면 이쪽으로 온다. 지니 계수는 0.051이므로 비교적 순도가 높다. 여기에 속하는 입력 샘플은 38개이고 이 중에서 첫 번째 유형이 1개, 두 번째 유형이 1개, 세 번째 유형이 37개라는 의미이다.



```
import graphviz    #https://graphviz.org/
import os
# 5. Graphviz로 고해상도 시각화 (수정된 부분!)
dot_data = export_graphviz(clf, out_file=None, feature_names=iris.feature_names,
                           class_names=iris.target_names, filled=True, rounded=True,
                           special_characters=True)
# PNG 파일 저장
output_file = "iris_decision_tree"
graph = graphviz.Source(dot_data)
graph.render(filename=output_file, format='png', cleanup=True) # .png + .pdf 생성
print(f"\n의사결정나무가 저장되었습니다:")
print(f"    → {os.path.abspath(output_file + '.png')}")
print(f"    → {os.path.abspath(output_file + '.pdf')}")
# 자동으로 이미지 열기 (Windows/macOS)
try:
    if os.name == 'nt': # Windows
        os.startfile(os.path.abspath(output_file + '.png'))
    else: # macOS
        os.system(f"open {os.path.abspath(output_file + '.png')}")
except:
    print("이미지 자동 열기 실패 → 수동으로 확인하세요.")
```

예제: 의사 결정 트리를 이용한 붓꽃 분류



Q & A

